

SwellDB: Dynamic Query-Driven Table Generation with Large Language Models

Victor Giannakouris
Cornell University
Ithaca, NY, USA
vg292@cornell.edu

Immanuel Trummer
Cornell University
Ithaca, NY, USA
it224@cornell.edu

Abstract

We present SwellDB, a data system that enables analytical query processing over tables generated dynamically with Large Language Models (LLMs). Typically, traditional database systems function under the closed-world assumption, allowing query execution over already existing tables. In contrast, SwellDB dynamically generates the required tables based on input SQL queries, and user-defined table definitions. Powered by an LLM, SwellDB integrates external data sources, including diverse datasets, database systems, and search engines to extract and blend relevant information into a structured table. Given an input query and schema, SwellDB intelligently selects data sources, synthesizes structured tables adhering to the specified schema, and makes them queryable through its execution engine. During the demonstration, we will showcase how SwellDB addresses data generation and integration challenges across domains such as web search and bioinformatics. Attendees can define custom schemas and execute SQL queries, gaining hands-on experience with SwellDB's table generation capabilities.

Video URL: <https://youtu.be/X1VOCVYb3Sc>

Keywords

Database, Large Language Models, Table Generation

ACM Reference Format:

Victor Giannakouris and Immanuel Trummer. 2025. SwellDB: Dynamic Query-Driven Table Generation with Large Language Models. In *Companion of the 2025 International Conference on Management of Data (SIGMOD-Companion '25)*, June 22–27, 2025, Berlin, Germany. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3722212.3725136>

1 Introduction

In this demo, we introduce SwellDB, a data system that follows a new architecture that enables query execution over dynamically generated tables. Unlike traditional systems which rely on previously stored data, SwellDB synthesizes tables on the fly based on input queries, utilizing Large Language Models (LLMs) to generate structured data according to user-defined prompts, schemas, or SQL queries. By efficiently integrating data from multiple sources, including LLMs, file formats, databases, and raw text such as web search results, SwellDB bridges the gap between *structured query execution and unstructured data retrieval*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGMOD-Companion '25, June 22–27, 2025, Berlin, Germany

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1564-8/2025/06

<https://doi.org/10.1145/3722212.3725136>

Closed-world Assumption. Traditional database systems operate under the closed-world assumption, where queries are executed on pre-existing, structured tables. Structured and queryable data offers several key advantages. First, database systems enable users to leverage SQL, a widely adopted and standardized language over the past decades, for executing analytical queries and retrieving precise results. Second, structured data facilitates seamless integration with external services, such as mobile applications and websites, through a wide range of connectors and APIs. However, while querying structured data is highly desirable, the vast majority of data resides in external semi-structured or unstructured sources. Consequently, end-users are often burdened with developing custom Extract-Transform-Load (ETL) workflows to create structured tables, enabling them to query this data using SQL.

Large Language Models. LLMs offer a compelling solution to the challenges of information integration and table generation. Trained on vast amounts of data embedded within their weights, LLMs excel at data generation and augmentation tasks. For example, given input datasets and a search engine, an LLM can determine how to utilize these data sources to generate a table with specific content and schema. LLMs can efficiently answer questions such as which code or SQL query to execute to retrieve a portion of the requested data from the input data sources, or which search queries to issue in order to retrieve data not available in the LLM or input sources. Additionally, LLMs can serve as preprocessors, transforming unstructured data (e.g., web-retrieved information) into structured formats that are queryable with SQL.

Taking these considerations into account, we pose the following question:

Given a set of input data sources, how can we leverage Large Language Models to dynamically generate tables of any content based on an input query?

We address this question by introducing a data system architecture that enables query execution over dynamically generated tables, according to the input query. Our system, SwellDB, leverages Large Language Models to tackle the table generation problem as follows. Given a table definition consisting of a prompt (describing the table content) and a schema (a set of attributes and their descriptions), SwellDB interacts with the LLM to compute a table generation plan. This plan specifies (1) which input data sources can be used to retrieve portions of the data, (2) how to utilize these data sources, and (3) which parts of the requested data are missing and need to be retrieved from the web. Furthermore, SwellDB employs the LLM as a data processing operator to transform unstructured data into a structured format, ensuring that extracted information can be seamlessly integrated into a unified table. As a result, SwellDB enables a paradigm shift from the closed-world to

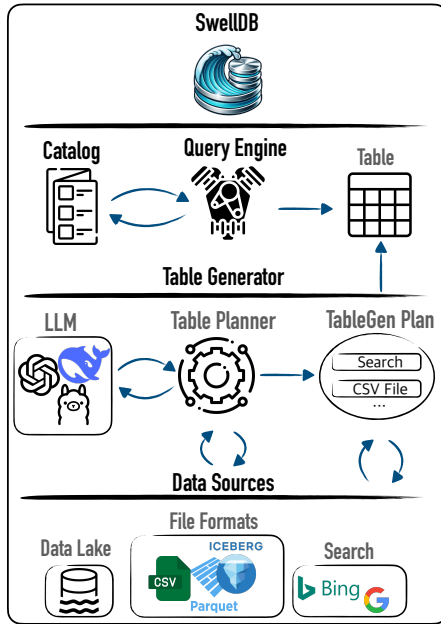


Figure 1: SwellDB Architecture

the open-world assumption in query execution. Instead of restricting queries to pre-existing tables, SwellDB dynamically generates tables on the fly, incorporating any relevant information based on the input query.

During the demonstration, attendees will be able to experiment with SwellDB's table generation capabilities over a wide range of use cases. We will provide a series of table generation scenarios, including tables that can generated solely from the LLM, the local data sources, the search engine, or combinations of all these. At the same time, users will be able to experiment using their own ad-hoc table definitions, and SQL queries.

2 Overview

This section presents an overview of SwellDB. In the following subsections, we will be discussing its architecture, the table generation process, as well as the query execution engine.

2.1 Architecture

Figure 1 illustrates SwellDB's architecture, which consists of the following components:

Catalog. The catalog maintains metadata for registered tables. SwellDB's catalog differs from traditional database systems, since SwellDB dynamically generates tables at runtime based on input queries rather than storing materialized tables. As shown in Figure 2, each registered table in the catalog comprises two parts: (1) a natural language prompt describing the table, and (2) attribute names with corresponding descriptions. Once a table is referenced, both of these parts are used in order to generate the table accordingly.

Table Generator. The Table Generator forms the backbone of SwellDB, comprising the *LLM API* and the *Table Planner*. The LLM

Create a new table

Enter the table name

genetic_mutations

Enter the table description

Create a table that contains genetic mutations, where each row contains the associated protein and disease.

Fields

Number of fields

3

	Field Name	Field Description
0	mutation	The mutation code
1	disease	The associated disease
2	protein	The associated protein

Figure 2: SwellDB Catalog

Plan Metadata

```
{
  "missing_columns": [...],
  "llm_columns": [...],
  "search_engine_columns": [],
  "dataset_columns": {...},
  "datasets": [...],
  "sql_query": "SELECT name, age, blood_type FROM patient WHERE age <= 30",
  "linking_column": [...]}

```

Table Generation Plan

```
{
  "join_key": "blood_type"
  "nodes": [
    0: "DatasetTable[schema={'blood_type': ''}, 'name': '', 'age': '', query='SELECT name, age, blood_type FROM patient WHERE age <= 30']"
    1: "SearchEngineTable[schema={'compatible_types': '', 'blood_type': ''}]"
  ]
}
```

Figure 3: Plan Metadata

API, implemented using LangChain¹, integrates seamlessly with

¹<https://www.langchain.com/>

widely used LLMs such as DeepSeek², OpenAI’s GPT³ and Ollama⁴. The Table Planner constructs a generation plan for each table definition from the catalog. This plan outlines the steps to create the table, interacting with the LLM as needed. For instance, the planner may query the LLM to determine whether a specific column can be generated from input datasets or requires direct LLM computation. The final table is constructed by executing the plan using the LLM for individual operations and the query engine for integration. Additional details are provided in Section 2.2.

Query Engine. The query engine supports SwellDB in two key ways. First, it acts as the primary interface for executing SQL queries. Second, it integrates intermediate results of the table generation pipeline into a unified table. Intermediate results are represented as DataFrames, and the query engine uses SQL operations, such as joins, to blend these results into the final table. SwellDB’s query engine is built on Apache DataFusion⁵, an embedded, in-memory query engine.

Data Source API. The Data Source API enables developers to add custom drivers for integrating external data sources with SwellDB. It supports two types of data sources:

- **Unstructured Data Sources:** These ingest diverse formats, such as text or HTML/XML, and rely on the LLM to transform data into structured DataFrames for integration.
- **Structured Data Sources:** These require predefined schemas (e.g., database systems, CSV, or Parquet). Structured sources are more efficient, as the planner can directly generate SQL queries for data extraction, bypassing additional transformation steps by the LLM.

2.2 Table Generation

The table generation process is the heart of SwellDB. In summary, the table generation is split into smaller operations by the table planner. Each plan operator is responsible for creating a subset of the table columns using the LLM and the input data sources as a DataFrame. The results of all operators are DataFrames that share a common key. In the final step, the individual outputs are joined using a generated SQL query using the Query Executor.

Planning. Once a table is referred either via the catalog or an SQL query, the table planner analyzes the table definition (content and attributes) and the connected input data sources. The output of this step is a table generation plan that describes *which* data sources to use and *how*. First, the planner issues a query to the LLM in order to obtain the table plan metadata (see Figure 3), which contains information like which columns can be created solely from the LLM (*llm_columns*), from the input data sources (*dataset_columns*), or which columns are not available neither by the LLM or the input data sources (*missing_columns*) and need to be retrieved from the web. For instance, if part of the data requested in the table’s content is contained in some of the input databases, the planner will use the LLM in order to generate the SQL code required to extract this information from that database, and it’s described as a *DatasetTable*. On the other hand, if part of the information is not included in any

of the input data sources, the planner will use the LLM in order to generate a number of *search queries* and will attempt to find the required information on the web, using the connected search engine data, defined as a *SearchEngineTable*. The result of this process will be a table generation plan which contains the steps required to generate each part of the table. For example, the table plan depicted in Figure 3 consists of a *DatasetTable* that extracts the attributes *name*, *age* and *blood_type* from the *patients* table depicted in Figure 4. The second operator is a *SearchEngineTable* that searches on the web for the compatible types of each blood type provided by the previous input dataset.

Materialization. This step executes the table generation plan obtained by the previous step, in order to create the final *table materialization*, which contains the actual rows. In the final step all operators are materialized as DataFrames, and they are integrated using a join query that is executed in the query engine.

2.3 Query Execution

In the previous section we discussed how tables can be generated when they are registered by the user. In this section we discuss SwellDB’s query executor, which allows users to freely execute SQL queries without the need to think about the underlying tables. SwellDB decides how to generate the required tables using the input data sources in order to generate a table with respect to the input query. For instance, Figure 5 depicts the execution of an SQL query that joins the *patients* table with the *type* table in order to generate a table that contains the compatible blood types for each patient. After issuing the query, the table planner decides to extract all the patient information from the patients CSV file by issuing an SQL query to extract all the patients with $age \leq 30$ (see Figure 3). Next, it creates the *type* table using a *SearchEngineTable* operator. Finally, it will create the final table depicted in Figure 5 by joining the *patient* table created using the CSV file, and *type* table created using the LLM and the web search results.

3 Demonstration

Our demonstration script consists of two parts: (1) The table generator and (2) the query engine. In both cases, users will be able to use either one of our scenarios, or define and create their own tables, or ad-hoc queries. Although users are welcome to come up with their own use cases, the following scenarios will be provided:

Scenario 1: LLM-Only Generation. In this scenario, we will provide a set of table definitions of tables that can be constructed using only the LLM. Use-cases like these include tables containing information that cannot change, and is already seen by the LLM. Examples can be a table that contains all the elected presidents in the US from 1990 to 2020, or a list of rock bands from the 80s.

Scenario 2: Dataset-Only Generation. This scenario will include definitions of tables that can be generated using one or more datasets. This scenario will demonstrate SwellDB’s ability to utilize the LLM in order to generate the required SQL queries in order to extract and combine data from one or more input data sources, and construct the final table by simply executing the queries locally.

Scenario 3: LLM & Dataset & Search Generation. In this scenario, the requested table will be partially available in one or more of the local datasets, but some of the columns will be missing and

²<https://www.deepseek.com/>

³<https://openai.com/>

⁴<https://ollama.com/>

⁵<https://datafusion.apache.org/>

Dataset Explorer

Available datasets

patient.csv			
	name	blood_type	age
0	Sophia Johnson	AB+	67
1	Sarah Brown	A-	40
2	James Garcia	B-	51
3	John Davis	B+	87

Figure 4: Dataset Explorer

Figure 5: Query Engine

will need to be retrieved. An example of this use case is the genetic mutations table depicted in Figure 2. For this table, both the mutations and the proteins are available in local datasets, but the associated disease has to be retrieved from the web. In this case, SwellDB will make use both of the LLM, the local datasets and the search results in order to generate the final table.

Scenario 4: LLM & Dataset. In this scenario, the requested table can be constructed partially using data from a local dataset, but a requested column can be generated using the LLM. An example of that use-case is a local CSV file that contains customer reviews for a specific product, and the requested table also contains a "sentiment" attribute. All of the aforementioned scenarios are applicable both in the table generator and query executor. As a result, users will be able to use our already existing, or provide their own datasets and define their ad-hoc tables or queries.

4 Conclusions and Related Work

Large Language Models (LLMs) have revolutionized data management, opening new research directions for integrating LLMs into database systems. Several works have explored how LLMs can be efficiently incorporated into query execution, with a predominant focus on integrating declarative languages with LLM invocations [5] for hybrid querying, as seen in systems like SWAN [9],

BlendSQL [3], Palimpzest [4], and LOTUS [8]. These systems optimize declarative querying by supporting tasks such as data augmentation (e.g., generating new columns) and enabling semantic operations like natural language joins and selections. However, they operate on pre-existing structured datasets. Galois [6] introduces an alternative paradigm where LLMs function as a storage layer for SQL execution. While this eliminates reliance on external databases, it remains constrained by the LLM’s internal knowledge and lacks integration with external structured or unstructured data sources. Beyond-the-database querying has also been explored in prior crowdsourced systems like CrowdDB [2] and DECO [7], though these approaches depend on user-provided data rather than autonomous LLM-based extraction. Evaporate [1] further advances LLM-powered data retrieval by generating extraction scripts for heterogeneous data lakes.

SwellDB distinguishes itself by introducing a dynamic table generation model that constructs relational structures on the fly, tailored to input SQL queries. Unlike systems that enhance existing tables, SwellDB synthesizes tables by intelligently selecting and blending data from multiple sources, including LLMs, structured databases, and web search results. This approach bridges the gap between structured query execution and unstructured data retrieval. Through our demonstration, we will showcase how SwellDB effectively leverages LLMs for dynamic table generation, enabling more flexible and expressive analytical querying.

Acknowledgments

This material is based upon work supported by the National Science Foundation under Award No. 2239326.

References

- [1] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avani Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language models enable simple systems for generating structured views of heterogeneous data lakes. *arXiv preprint arXiv:2304.09433* (2023).
- [2] Amber Feng, Michael Franklin, Donald Kossmann, Tim Kraska, Samuel R Madden, Sukriti Ramesh, Andrew Wang, and Reynold Xin. 2011. Crowddb: Query processing with the vldb crowd. (2011).
- [3] Parker Glenn, Parag Pravin Dakle, Liang Wang, and Preethi Raghavan. 2024. BlendSQL: A Scalable Dialect for Unifying Hybrid Question Answering in Relational Algebra. *arXiv preprint arXiv:2402.17882* (2024).
- [4] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. [n. d.]. Palimpzest: Optimizing AI-Powered Analytics with Declarative Query Processing. ([n. d.]).
- [5] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E Gonzalez, Ion Stoica, and Matei Zaharia. 2024. Optimizing llm queries in relational workloads. *arXiv preprint arXiv:2403.05821* (2024).
- [6] Paolo Papotti. 2024. Large Language Models as Storage for SQL Querying. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 5657–5658.
- [7] Aditya Ganesh Parameswaran, Hyunjung Park, Hector Garcia-Molina, Neoklis Polyzotis, and Jennifer Widom. 2012. Deco: declarative crowdsourcing. In *Proceedings of the 21st ACM international conference on Information and knowledge management*. 1203–1212.
- [8] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. Lotus: Enabling semantic queries with llms over tables of unstructured and structured data. *arXiv preprint arXiv:2407.11418* (2024).
- [9] Fuheng Zhao, Divyakant Agrawal, and Amr El Abbadi. 2024. Hybrid Querying Over Relational Databases and Large Language Models. *arXiv preprint arXiv:2408.00884* (2024).